RMIT UNIVERSITY

School of Computing Technologies

ISYS1102/ 3479 Database Applications

Week 2: Tute/Lab – Building an efficient database backend for an application

Semester 1 2026

1 Objective

The objective of this tute/lab session is to learn basics of building an **efficient** database backend for an application, specifically:

- analyse the requirements outlined in the problem description;
- develop a conceptual model to assist you with the design of the database backend required for the system;
- compute estimated table and index sizes and the estimated growth for extremely large tables;
- identify and implementing efficient storage strategies for extremely large tables;
- identify and implementing efficient partition strategies for extremely large tables;
- write efficient queries on extremely large tables and describe the query plans that query optimiser would likely be using and explain how a cost-based query optimiser would execute such query plans;
- write stored procedures in T-SQL to automate common tasks on Microsoft SQL Server.

IMPORTANT: While most of the related concepts are not yet covered in the lectorials, we can start with steps 1 – 3 this week. The rest of the activities can be completed over coming weeks _after_ corresponding concepts are taught in the lectorials.

The tasks in Assignment 1 are closely related to this activity. So, in-class discussions in tute/lab sessions will reinforce the required skills for the successful completion of the Assignment 1.

2 Preparation Tasks

[For those who missed out Week 1 tute/lab session.]

If you did not attend the Week 1tute/lab session, you must make sure you have set up VS Code in your computer to access SQL Server. Please refer to Week 1 Activity Sheet.

3 Activities

3.1 Activity 1: Completion of SQL Revision Questions from Week 1 Lab/Tute Session

As it is extremely important that you have a very good comprehension in SQL Programming and be able to use it to write complex queries, you should complete the remaining parts of Week 1 Revision exercises this week.

Among the advanced SQL skills, I must emphasise that you should be able to write queries using:

- Nested queries
- Sub-Selects
- Queries using IN, EXISTS
- Different types of JOINs, including self-joins
- Grouping and grouped aggregation -- GROUP BY, HAVING
- Set operations

3.2 Activity 2: Building an efficient database backend for an application

In your first-year database concepts courses, you developed skills in analysing user specifications for a database-driven application, design the conceptual design, convert a conceptual model into a relational database model and build a database using the final physical database design. You also develop skills in writing SQL statements to carry out various CRUD tasks.

However, while you have given special attention to good design practices such as normalisation and integrity of data, it is unlikely that sufficient attention was given to the efficiency characteristics of the underlying database design. In this activity, you will do a fully-fledged (from the user requirements to all the way to final database deployment) while focussing on the efficiency characteristics at each stage of your database design.

Problem Description

Victoria's public transport ticketing system (MyKi) is aging and up for a replacement. The governing body – Public Transport Victoria (PTV) – has called for tenders for a replacement system. The multi-national infrastructure development company you just joined won the contract. You are now in the project team, with your database skills, and tasked to design the database back-end.

In the Phase 1 of the project, it is not expected to change the basic ticketing guidelines (such as zones, MyKi Money and MyKi Pass options, or the pricing). In this phase, you will simply replace the aging infrastructure, including the substantially slow database infrastructure. In fact, after your initial investigation on the performance issues in the current system, you found out that database infrastructure is one of the bottlenecks in the system and it slows down the operation of the whole system.

IMPORTANT: You may not have sufficient background knowledge or theoretical underpinnings for some sections of the following tasks. Complete the ones that you can do today with your current database knowledge and skills set. As we progress with more database optimisation strategies, we will, during next 4 weeks, re-visit the tasks we have left behind and complete them.

3.2.1 Task 1

Discuss with other students in your table, the requirements for the back-end database for the MyKi system. In your discussion, consider the current operational requirements and how the back-end database would support such operational requirements.

At the end of your deliberations, produce a data model for this application, using Entity-Relationship diagram. You are free to use any Entity-Relationship notation, however, UML notation is preferred (most of the pre-req courses at RMIT do use UML notation, so, you must be familiar

with this notation. If you need any assistance with the notation, you may refer to: https://vertabelo.com/blog/uml-notation/ UML is a less ambiguous notation compared to other Entity-Relationship notations and has many similarities with the notations used in Object-Oriented Software Design.

3.2.2 Task 2

Identify the tables, in your final schema, that are expected to be extremely large and are expected to grow over time.

For each of these tables:

- Describe the expected record size (in bytes or kilobytes), the estimated initial table size, and estimated table size after 10 years of use.
- Describe a suitable storage strategy for such tables.

The number of bytes used for different data types differ among different DBMSes. This document outlines the number bytes used in common data types used in Microsoft SQL Server: https://www.w3schools.com/sql/sql_datatypes.asp

Scroll down to the SQL Server Data Types section.

The Victoria state government public data repository has the monthly public transport patronage data. A copy of this data set is available on Canvas (Week 2 module) or, directly downloadable from: https://discover.data.vic.gov.au/dataset/monthly-public-transport-patronage-by-mode

As of now, MyKi ticketing system is used across all modes of transport. So, you have to aggregate data across all columns to get the total MyKi patronage.
As outlined in the following chart, Victoria population is 6.8 million in 2023. it increases to 10.3 million by 2051. Source:
https://www.planning.vic.gov.au/__data/assets/pdf_file/0022/703453/DTP0552-Victori-in-Future-2023-report.PDF

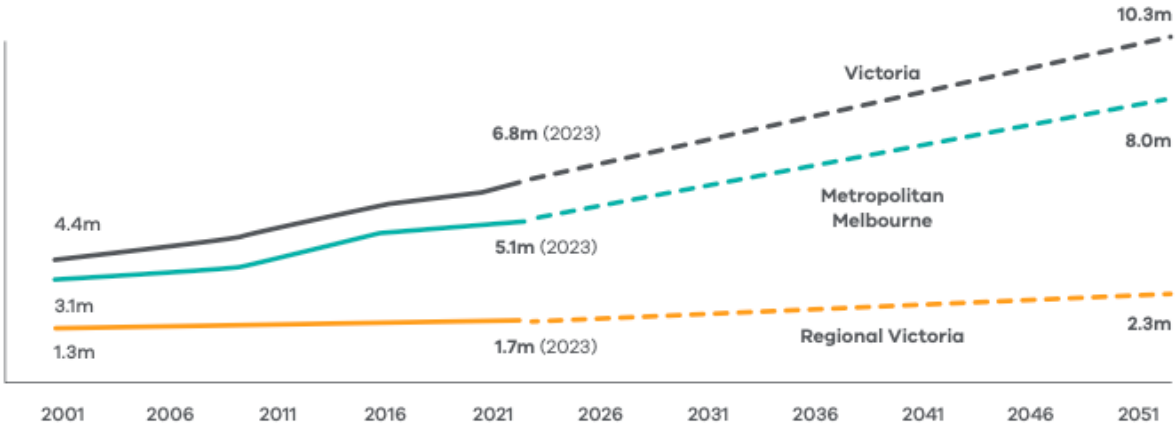

Figure 1 – Past and projected population by major regions 2001 to 2051.

You can assume the public transport patronage will also grow at or above the rate of population growth. The future planned public transport infrastructure projects, such as Metro Tunnel

(https://bigbuild.vic.gov.au/projects/metro-tunnel), Suburban Rail Loop (https://bigbuild.vic.gov.au/projects/suburban-rail-loop), Eastern Busway (https://bigbuild.vic.gov.au/projects/roads/eastern-freeway-upgrades/eastern-busway) will in fact have a big impact on future patronage. So, in your calculations on the future growth in your data, you should give sufficient provisions for such growth (on top of population growth).

3.2.3 Task 3

Identify three most common data requests that would likely to occur.

In the normal day-to-day operation of the MyKi system, one of the most common transactions is Touch-On/Off of tickets at the scanners (NFC Readers). Each Toch-On or Touch-Off would invoke a number of SQL queries that is being sent to the database back-end you designed.

Write down the queries that would be executed with each Touch-On or Touch-Off.

What are the other most common operations in MyKi system? Identify the underlying database queries invoked by these operations.

For each of the queries:

- Produce the SQL query, but do not include the results set.
- Identify what indexes would help. Identify the type of index and columns that are used to build these indexes (justify your design).
- Show the SQL commands for building these indexes in SQL Server.
- Show the query execution plans both before the index is added and after adding the index.

3.2.4 Task 4

Describe a suitable partition strategy for extremely large tables you identified in Task 1. Include details of the partitioning type and which columns/key should be used. You must include the SQL DDL statements used to implement your partition strategy.

You must justify your design decisions. Include details about which of the above queries it will improve the performance of and how it helps with concrete examples. (You must explain in clear terms – such as partition pruning, partition joins, and parallel SQL, applicable to each of these queries.)